

PFC AI Design Agent

Architecture, Workflow and Framework

How the tool is built, what each agent does, where the designer decides, and how simulation is used as an independent check.

*Generated 2 August 2026 | describes the codebase at commit **64fcfb4** (C190)*

1. What this tool is

The PFC AI Design Agent takes a power-supply specification and produces a complete, traceable design for a Power Factor Correction stage — magnetics, DC-bus capacitor, control loops, semiconductors, input protection and EMI filter — together with an approximately 190-page engineering report that shows every calculation.

It is deliberately **not** an autonomous agent. Every stage produces a recommendation with its reasoning, and the designer approves, overrides or rejects it before anything advances. The tool does the arithmetic, holds the vendor data, and refuses to hide an assumption; the engineering judgement stays with the designer.

Three principles the codebase is built on

- **One engine per quantity.** If two chapters print the same number they call the same function. Where that rule was broken it produced real defects — a current density that read 4.17 in one place and 4.12 in another, a capacitor loss that differed by several times between chapters.
- **No hidden defaults.** A value that is not supplied is printed as **DATA MISSING**, never silently replaced. A missing OVP threshold must not look like a justified voltage margin.
- **Gates release, never selection.** An unverified or missing datasheet value can stop a release sign-off, but it must never make every part un-selectable. The designer can always proceed with a documented risk.

2. End-to-end workflow

The diagram below is the whole tool on one page. Read it as four horizontal lanes: what the **designer** decides, what the **GUI** shows, which **engine** computes it, and which **data** it draws on. Work flows left to right; every vertical arrow crossing into the designer lane is a point where the tool stops and waits.

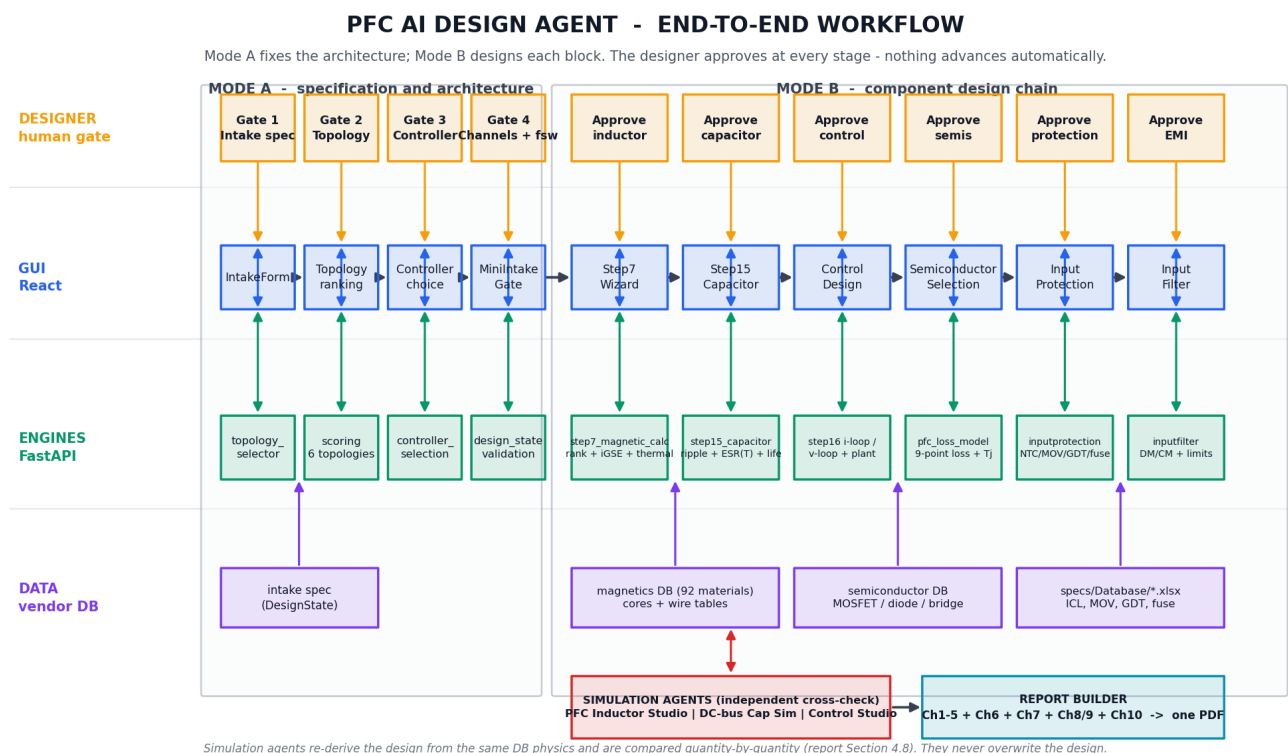


Figure 1 — End-to-end workflow. Mode A fixes the architecture (4 gates); Mode B designs each block (6 approvals).

3. Mode A — specification and architecture

Mode A answers “what are we building?” It does not size a single component. Its job is to lock the decisions that everything downstream depends on, so that no later stage quietly re-opens them.

Gate	What the tool does	What the designer decides
1 — Intake	Collects the electrical specification: input range, output bus, power at low and high line, hold-up time and minimum bus voltage, ripple, ambient, cooling, compliance class.	Enters and confirms the specification. This becomes DesignState, the contract every later step reads.
2 — Topology	Scores six candidate topologies against five weighted criteria and ranks them.	Accepts the recommendation or picks another. The choice is locked for all later chapters.
3 — Controller	Proposes analog or digital control and a reference IC, with the trade-offs.	Selects the control architecture. Drives the whole of Chapter 6.
4 — Channels and f_{sw}	Derives the interleaving benefit, switching-frequency trade-off and crest ripple ratio.	Sets channel count, switching frequency and ripple ratio — the three numbers that size the inductor.

4. Mode B — the component design chain

Each block follows the same shape: **compute the requirement** → **screen the catalogue** → **the designer selects** → **re-verify on the actual part**. That ordering matters. An earlier version named the chosen part before deriving the requirement, which made the reasoning circular; Chapters 8 and 9 were restructured specifically to fix that.

Stage	Engine	What it computes	Designer decision
Inductor	step7_magnetic_calc	Per-point required inductance from the ripple ratio, bias-aware turns convergence, iGSE core loss (crest and cycle-averaged), copper loss, two-node thermal model. Ranks the core catalogue.	Picks a core and winding from the ranked list; may re-run with different constraints.
DC-bus capacitor	step15_capacitor	Required capacitance from hold-up, ripple current per capacitor, vendor-implied ESR(T), self-heating and lifetime. Now also the $\pm 20\%$ tolerance corners.	Selects the bank; sees pass/fail at nominal and at both tolerance corners.
Control loops	step16 current / voltage loop	Plant model at nine operating points, current-loop and voltage-loop compensators, crossover, phase and gain margin, component values snapped to E-series.	Sets crossover targets and confirms the compensator; seven confirm-gated screens.
Semiconductors	pfc_loss_model	Bridge, MOSFET and boost-diode loss by mechanism (conduction, switching, E_{oss} , recovery, gate drive) across nine points, plus the thermal network and junction temperatures.	Chooses each device from a loss-ranked shortlist.
Input protection	inputprotection (NTC / MOV / GDT / fuse)	Inrush limiting, surge clamping to IEC/EN 61000-4-5, bypass-relay timing and a six-gate fuse screen with I ² t coordination.	Selects NTC, MOV, GDT and fuse; resolves any release-blocking gate.
EMI filter	inputfilter	Differential- and common-mode attenuation against the chosen limit line, component selection and leakage check.	Confirms the filter topology and committed Y-capacitance.

5. Software framework

SOFTWARE FRAMEWORK - WHAT RUNS WHAT

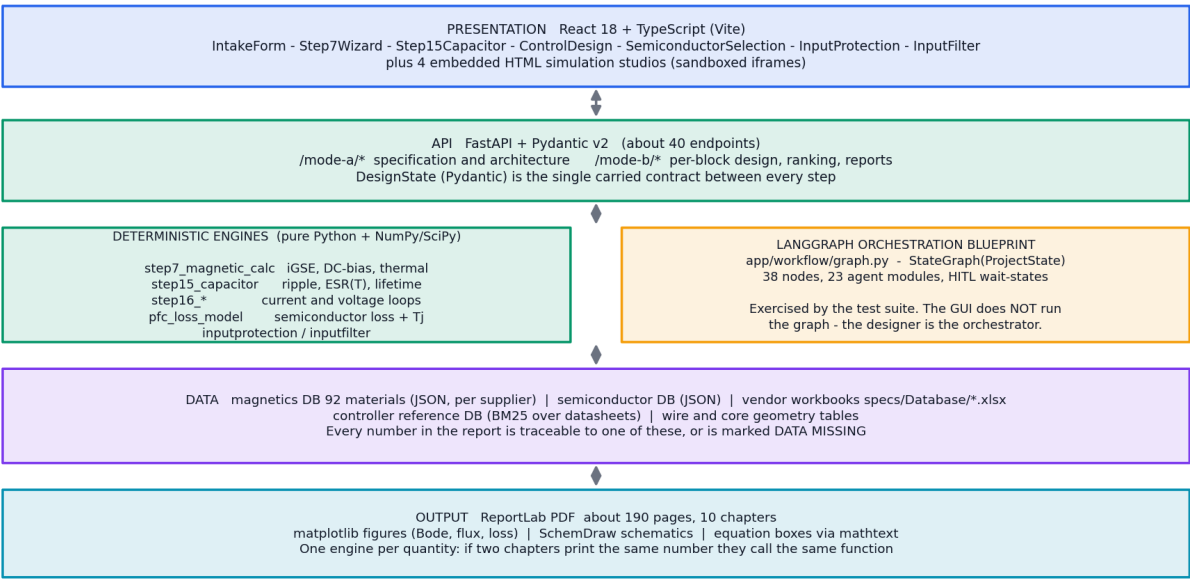


Figure 2 — Layer stack. Note the distinction between the deterministic engines that run in production and the LangGraph blueprint.

5.1 What actually runs when you use the tool

The production path is a **React front end calling FastAPI endpoints directly**, one stage at a time, with the designer as the orchestrator. There is no autonomous planner deciding what to do next — the sequence is the designer clicking through the stages, and each click is an explicit HTTP call to a deterministic Python engine.

Every calculation engine is **ordinary Python with NumPy and SciPy**. No large language model is involved in any number that reaches the report. This is a deliberate design choice: the outputs must be reproducible and auditable, and a designer must be able to follow the arithmetic in the report and get the same answer by hand.

5.2 LangGraph — and an honest word about it

The repository contains a **LangGraph** orchestration graph at `app/workflow/graph.py`: a `StateGraph` over a `ProjectState` with 38 registered nodes, conditional routing, and explicit human-in-the-loop wait states (`WAIT_TOPOLOGY`, `WAIT_CONTROLLER`, `WAIT_MODE_B`) that suspend the graph until a designer decision arrives. Twenty-three agent modules sit behind those nodes.

It is important to be accurate about its status: **the graph is currently exercised by the test suite, not by the GUI**. `build_graph()` is called from tests only. It is best understood as the orchestration blueprint — the design for how these agents would run autonomously, kept working and tested — while the shipping product routes the same engines through explicit endpoints under designer control.

That is a reasonable place to be for a design tool whose output has to be defensible. It also means the migration path is open: the engines are already isolated behind clean interfaces, so moving a stage onto the graph is a routing change, not a rewrite.

5.3 The technology list

Layer	Technology	Role
Front end	React 18, TypeScript, Vite	Seven design screens plus four embedded simulation studios.
API	FastAPI, Pydantic v2	About 40 endpoints. Pydantic models validate the design state at every boundary.

Layer	Technology	Role
Orchestration	LangGraph 0.2	38-node StateGraph with HITL wait states. Test-exercised blueprint (see 5.2).
Computation	NumPy, SciPy	All physics: iGSE core loss, waveform integration, control-loop analysis, thermal networks.
Reporting	ReportLab, matplotlib, SchemDraw	PDF generation, Bode and flux figures, circuit schematics.
Data	JSON material DB, Excel vendor workbooks, openpyxl	92 magnetic materials, semiconductor and protection catalogues.
LLM	Anthropic SDK	Available for datasheet extraction and reference search. Not in any calculation path.

6. Simulation and independent verification

Simulation in this tool is a **cross-check, not a design step**. The simulation agents never write back into the design. Their purpose is to re-derive the same quantities by a different route and expose disagreement.

Simulation agent	What it does	How it is used
PFC Inductor Studio (embedded, per-candidate)	Full inductor model — iGSE waveforms, flux and current over the line cycle, DC-bias sweep, 3-D winding view.	The designer opens it on any candidate to inspect the design interactively before approving.
Field engine cross-check (Python, automatic)	An independent implementation fed the same database physics, run on the approved design.	Compared quantity-by-quantity against the sizing engine; the result is printed in report Section 4.8 as a pass/review verdict.
DC-bus capacitor simulation	Two-band ripple and lifetime model for the bank, with a $\pm 20\%$ capacitance selector.	Confirms Chapter 5 ripple, hold-up and life, and lets the designer see the tolerance corners.
Control studio	Interactive Bode and stability exploration for the compensator.	Used while choosing crossover frequencies before the control design is committed.

The cross-check earns its keep. It is deliberately allowed to disagree: its own standalone acceptance applies stricter internal checks than the quantity-by-quantity comparison, and the report presents that as informational rather than as a verdict on the design, so a 6/6 agreement is never reported alongside a contradictory rejection.

7. Where the numbers come from

Traceability is the point of the report. Every material and thermal figure carries its provenance: supplier, catalogue revision and page, the temperature each curve is valid at, and which of two candidate models is the design model. Where the engine had to estimate a parameter from the scalars available, it says so and says what it estimated it from.

- **Chapter 3.2.6** — material provenance: supplier, catalogue page, loss-curve and DC-bias sources, B_{sat} temperature basis.
- **Chapter 4 Table 4.6b** — thermal model provenance, including the fact that the temperature rise takes no credit for forced air even when the specification says fan-cooled.
- **Chapter 7 Table 7.2d** — which semiconductor parameters are datasheet values and which the engine estimated.

The same discipline drives the open-items register: anything the tool cannot close by calculation — a missing datasheet column, an absent site value, a decision only the designer can make — is recorded rather than assumed away.